

ThePremArchive — arañadores Premier League

Documentación del proyecto (portada, necesidad de información, políticas, estadísticas, URLs semilla): [documentacion.md](#).

Este archivo es solo instrucciones técnicas para correr el código.

Estructura

- [common/](#) — módulos compartidos por ambos arañadores: políticas ([config.py](#), [urls.py](#), [extract.py](#), [dedup.py](#)) y almacenamiento ([store.py](#), [bitacora.py](#)). Define qué se descarga y por qué.
- [vanilla_spider_premierleague/](#) — arañador propio en Python (hilos, cola propia, descarga con [requests](#)). Define cómo se arma el motor de arañado.
- [scrapy_premierleague/](#) — arañador con Scrapy. Reusa [common/](#) tal cual; solo cambia el motor de arañado.

Instalación

```
pip install -r requirements.txt
```

Arañador propio (Python desde cero, hilos)

Correr **desde la raíz del repo**:

```
python -m vanilla_spider_premierleague crawl --target-gb 10 -w 48 --delay 1 --per-domain 3 -d 6
python -m vanilla_spider_premierleague status      # progreso y estadísticas
python -m vanilla_spider_premierleague reset      # tras un corte, devuelve a la cola las URLs en vuelo
```

Detalle de cada política y dónde vive en el código: ver [documentacion.md](#).

Arañador con biblioteca (Scrapy)

Correr desde cualquier carpeta (las rutas de salida están ancladas a la raíz del repo, no a la CWD):

```
cd scrapy_premierleague
scrapy crawl premierleague
```

Para una prueba corta antes de dejarlo corriendo horas:

```
scrapy crawl premierleague -s CLOSESPIDER_TIMEOUT=60 -s CLOSESPIDER_ITEMCOUNT=100
```

La diferencia real entre ambas implementaciones es el motor de arañado (hilos propios vs. el scheduler asíncrono de Scrapy) — la lógica de qué descargar, filtrar y deduplicar es literalmente el mismo código ([common/](#)).

Repositorio compartido

Ambos arañadores escriben al mismo lugar, sin importar cuál corras:

- [repo/](#) — el repositorio de texto plano final ([.txt](#) por documento, nombrado por hash de contenido)
- [data/crawl.db](#) — SQLite con metadatos y el índice de deduplicación
- [logs/bitacora.jsonl](#) (propio) y [logs/bitacora_scrapy.jsonl](#) (Scrapy) — bitácora de cada URL visitada

Como el nombre de archivo es el hash del contenido, si los dos arañadores bajan la misma noticia, no se duplica en el repositorio — y el pipeline de Scrapy consulta la misma base de datos para descartar duplicados y casi-duplicados contra lo que ya bajó el arañador propio (y viceversa).

```
python -m vanilla_spider_premierleague status # sirve para ver el total  
combinado de ambos
```